

2 Проектирование базы знаний интеллектуальной системы

2.1 Общая архитектура интеллектуальной системы и место базы знаний

Разрабатываемая система представляет собой RAG-справочную систему на графовой базе знаний Neo4j, предназначенную для предоставления справочной информации по технологиям искусственного интеллекта на основе структурированного контекста, извлекаемого из графа знаний. Система строится как совокупность нескольких взаимосвязанных подсистем: пользовательского интерфейса, серверной логики, подсистемы извлечения знаний, графовой базы знаний и модуля генерации ответа на основе большой языковой модели.

Пользовательский интерфейс обеспечивает ввод естественно-языкового запроса и отображение полученного результата. Серверная логика выполняет координацию основных этапов обработки запроса: принимает запрос от интерфейса, передаёт его в подсистему извлечения знаний, получает структурированный контекст, после чего формирует обращение к модулю генерации ответа. Подсистема извлечения знаний является промежуточным звеном между серверной логикой и хранилищем знаний: она определяет тип запроса, выбирает подходящий шаблон обращения к графовой базе, выполняет Cypher-запросы к Neo4j и формирует набор фактов и связей, релевантных вопросу пользователя.

Ключевое место в рассматриваемой архитектуре занимает база знаний. Именно она хранит формализованное описание предметной области в виде сущностей и отношений между ними. В отличие от неструктурированного текстового хранилища, графовая модель позволяет явно представлять семантические связи между моделями, методами, задачами, концептами и фреймворками. Это даёт возможность не только выполнять поиск по именам объектов, но и извлекать связанный контекст, например определять, для каких задач используется модель, на каких методах она основана и в каком программном окружении может быть реализована.

С точки зрения общей архитектуры система функционирует следующим образом. Пользователь формулирует запрос через интерфейс. Далее запрос передаётся в серверную часть, которая инициирует получение контекста из базы знаний. Подсистема извлечения знаний обращается к Neo4j, получает релевантный подграф, после чего возвращает структурированный результат серверной логике. Затем серверная часть совместно с вопросом пользователя передаёт этот контекст в LLM-модуль, который формирует итоговый ответ. Таким образом, база знаний выступает не изоли-

рованным хранилищем, а центральным источником структурированного контекста для всей интеллектуальной системы.

Схема верхнего уровня, отражающая состав основных компонентов системы и место базы знаний в архитектуре, приведена на рисунке 2.1.

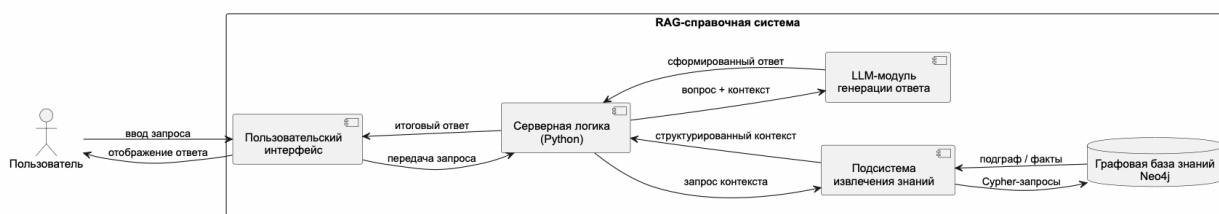


Рисунок 2.1 – Диаграмма компонентов RAG-справочной системы

Представленная архитектура соответствует распределению ролей в коллективном проекте. Общая логика функционирования системы является единой для всей команды, однако в рамках данной пояснительной записки основной акцент сделан на проектировании базы знаний, логике её организации и взаимодействии с подсистемой извлечения знаний.

2.2 Выбор формализма и логической основы

В качестве основного формализма представления знаний выбрана графовая модель, реализуемая средствами графовой системы управления базами данных Neo4j. Данный выбор обусловлен спецификой предметной области. Технологии искусственного интеллекта образуют множество взаимосвязанных сущностей, между которыми существуют отношения типа «является разновидностью», «используется для», «основана на», «реализована в», «связана с» и другие. Для такой предметной области графовое представление оказывается более естественным и наглядным, чем табличная реляционная модель, поскольку позволяет моделировать не только сами объекты, но и семантику их связей как самостоятельные элементы структуры данных.

В проекте графовая база знаний рассматривается как ядро механизма извлечения контекста. Узлы графа представляют основные сущности предметной области, а ориентированные отношения задают содержательные связи между ними. В результате можно формировать гибкие запросы к данным: находить соседние сущности, выявлять цепочки связей, извлекать подграф по заданному понятию и выполнять сравнение объектов по связанным с ними задачам, методам и инструментам реализации.

Дополнительным аргументом в пользу выбранного формализма является наличие в Neo4j языка Cypher, который предоставляет достаточно выразительные средства для поиска по узлам и отношениям, фильтрации, агрегации и извлечения связанных объектов. Это позволяет использовать единую логическую основу как для хранения знаний, так и для их последующей машинной обработки.

Синтаксически база знаний строится из двух основных типов элементов: узлов и отношений. Каждый узел принадлежит определённому типу сущности и содержит набор атрибутов. В текущей реализации используются типы **Model**, **Concept**, **Task**, **Method** и **Framework**. В качестве базового идентификатора во всех указанных типах используется атрибут **name**. Для узлов типа **Concept** дополнительно хранится атрибут **description**, а для узлов типа **Model** — атрибут **type**.

Семантически узлы интерпретируются следующим образом:

- **Model**, конкретная модель или семейство моделей искусственного интеллекта;
- **Concept**, обобщённое понятие или концепт предметной области;
- **Task**, задача, для решения которой применяется модель;
- **Method**, метод, механизм или приём, связанный с моделью либо концептом;
- **Framework**, программная среда или библиотека реализации.

Отношения задают семантические связи между сущностями. В текущем прототипе используются отношения **IS_A**, **USED_FOR**, **USES**, **IMPLEMENTED_IN** и **RELATED_TO**. Связь **IS_A** используется для отнесения модели к определённому концептуальному классу. Связь **USED_FOR** связывает модель с задачами её применения. Связь **USES** отражает использование метода или концепта. Связь **IMPLEMENTED_IN** фиксирует программную среду реализации. Связь **RELATED_TO** применяется для установления ассоциативных и предметно близких отношений между концептами и методами.

В качестве правил корректности приняты следующие соглашения:

- каждый тип узлов использует атрибут **name** в качестве уникального идентификатора;
- имена сущностей должны быть семантически однозначными и не дублироваться в пределах одного типа;
- отношения создаются только между совместимыми типами узлов;

- извлечение контекста осуществляется только на основе заранее определённых шаблонов Cypher-запросов.

Таким образом, логическая основа проектируемой базы знаний строится на сочетании графового представления объектов предметной области и набора семантически интерпретируемых отношений между ними. Такой подход обеспечивает достаточную выразительность модели и удобство её последующего расширения.

2.3 Концептуальная модель предметной области

Концептуальная модель предметной области определяет, какие сущности являются базовыми для справочной системы и каким образом между ними устанавливаются содержательные связи. В рамках проектируемой базы знаний предметная область ограничена технологиями искусственного интеллекта, прежде всего моделями, методами, концептами, прикладными задачами и программными фреймворками.

Центральным элементом модели выступает сущность **Model**, описывающая конкретные модели искусственного интеллекта. Для каждой модели может быть указан концепт, к которому она относится, одна или несколько задач применения, набор используемых методов и один или несколько фреймворков, в которых она может быть реализована. Сущность **Concept** предназначена для формализации более общих понятий предметной области. Концепты могут быть связаны друг с другом отношениями использования и предметной близости, а также могут быть связаны с методами. Сущность **Task** описывает прикладные задачи, такие как классификация, генерация текста или анализ изображений. Сущность **Method** представляет методы и механизмы, лежащие в основе моделей и концептов. Сущность **Framework** фиксирует используемые программные средства реализации.

Выбранная структура концептов поддерживает основные целевые сценарии системы. Во-первых, она позволяет отвечать на вопросы о назначении модели. Во-вторых, она обеспечивает возможность получения сведений о том, какие методы лежат в основе модели. В-третьих, за счёт явных связей между концептами становится возможным расширенное извлечение контекста, когда ответ формируется не только по одному узлу, но и по связанным с ним понятиям.

Концептуальная модель предметной области представлена на рисунке 2.2.

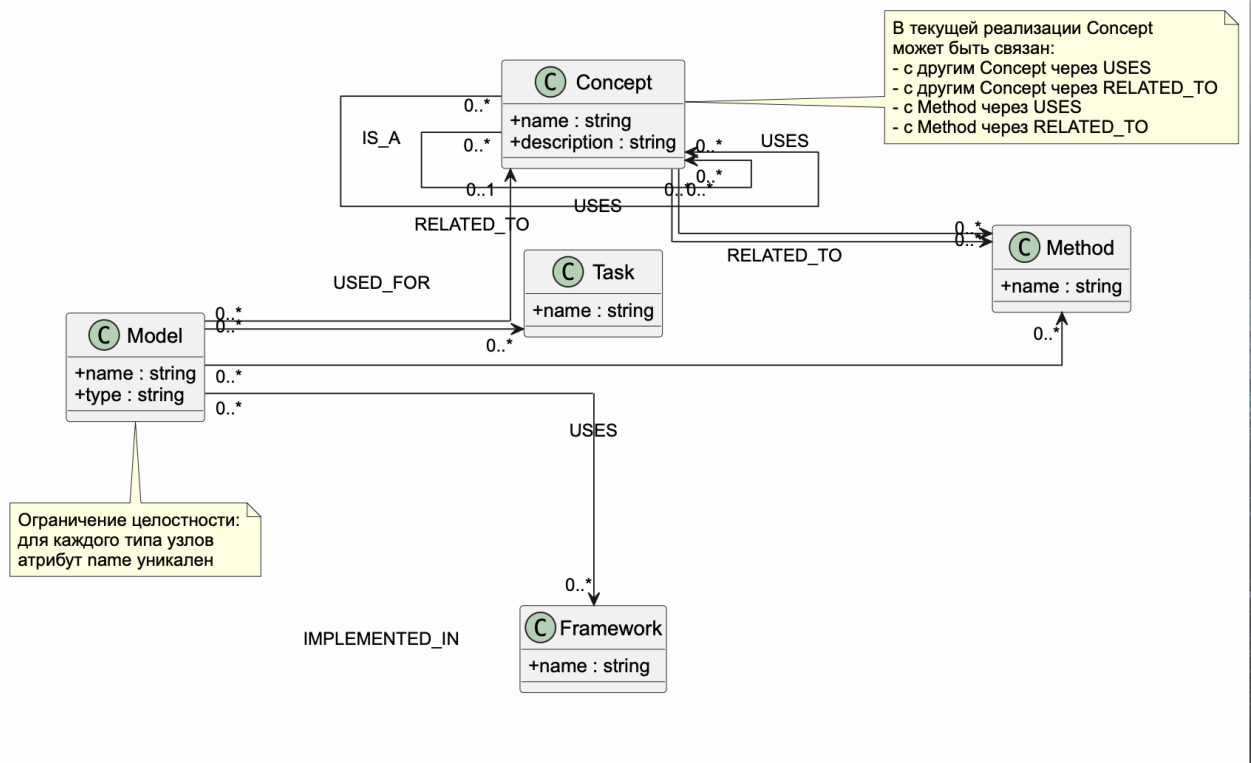


Рисунок 2.2 – UML-диаграмма классов как концептуальная модель сущностей и связей базы знаний

Следует отметить, что приведённая UML-диаграмма используется не как модель программных классов, а как концептуальное средство представления типов сущностей и допустимых связей графовой базы знаний. Такой способ описания предметной области удобен на этапе проектирования, поскольку позволяет формализовать состав сущностей и их роль в процессе извлечения знаний.

2.4 Логическая и физическая структура базы знаний

Логическая структура базы знаний организована по принципу разделения предметной области на несколько взаимосвязанных смысловых слоёв. Первый слой образуют сущности-объекты, непосредственно отражающие содержание предметной области: модели, задачи, методы, концепты и фреймворки. Второй слой образуют отношения между этими сущностями. Третий слой составляют ограничения целостности и правила интерпретации связей, которые обеспечивают единообразие представления знаний.

В логическом смысле узлы **Model** выступают основными точками входа в граф, поскольку значительная часть пользовательских запросов ориен-

тирована именно на конкретные модели. Узлы **Concept** формируют понятийный каркас базы знаний. Узлы **Task**, **Method** и **Framework** играют роль уточняющих и связывающих компонентов, которые раскрывают прикладное назначение модели, её методическую основу и программное окружение.

Для работы с базой знаний сформирован набор шаблонов запросов на языке Cypher. В текущем прототипе предусмотрены операции поиска сущности по имени, поиска по подстроке, извлечения описания концепта, получения соседних узлов, выбора задач модели, методов модели, фреймворков модели, а также сравнения двух моделей по связанным задачам и методам. Это позволяет рассматривать логическую структуру базы знаний не только как хранилище данных, но и как основу процедур извлечения контекста.

Физически база знаний реализуется в Neo4j и хранится в виде графа, состоящего из узлов, отношений и их свойств. Инициализация структуры выполняется с помощью Cypher-скриптов. На текущем этапе проекта используются три основных файла: `seed.cypher`, `constraints.cypher` и `queries.cypher`.

Файл `seed.cypher` содержит исходное наполнение графа: создание базовых узлов и отношений между ними. Файл `constraints.cypher` задаёт ограничения целостности, прежде всего уникальность атрибута `name` для каждого используемого типа сущностей. Файл `queries.cypher` содержит набор шаблонов запросов, применяемых подсистемой извлечения знаний при обработке пользовательских обращений.

Такой способ организации физических данных обеспечивает ряд преимуществ. Во-первых, структура хранения остаётся прозрачной и легко контролируемой. Во-вторых, уменьшается дублирование информации за счёт повторного использования узлов и связей. В-третьих, при расширении базы знаний не требуется изменять общую архитектуру хранения: достаточно добавлять новые узлы, отношения и при необходимости новые шаблоны запросов.

2.5 Моделирование видов знаний

Основу проектируемой базы знаний составляют декларативные знания, то есть факты о сущностях предметной области и отношениях между ними. К таким знаниям относятся сведения о том, что определённая модель относится к конкретному концепту, используется для решения определённой задачи, опирается на тот или иной метод и реализуется в соответствующем фреймворке. Аналогичным образом декларативно описываются связи между концептами и методами.

Преимущество такого представления состоит в том, что база знаний хранит не только изолированные факты, но и контекст их использования. Например, знание о том, что модель относится к определённому кон-

цепту, само по себе полезно, но в сочетании со связями `USED_FOR`, `USES` и `IMPLEMENTED_IN` оно образует более содержательный фрагмент графа, который может быть использован при генерации ответа.

Хотя ядро базы знаний является декларативным, в проекте также присутствует процедурная составляющая. Она выражается не в виде продукционных правил классического экспертного типа, а в виде алгоритмов и шаблонов обработки запросов, определяющих, каким образом из графа должны извлекаться нужные факты. К процедурным знаниям относятся:

- правило определения типа запроса;
- правило выбора соответствующего шаблона Cypher-запроса;
- правило выполнения fallback-поиска при отсутствии точного совпадения;
- правило формирования структурированного контекста на основе найденного подграфа;
- правило возврата пустого результата при отсутствии релевантных данных.

Указанные процедуры отражают рабочую логику взаимодействия с графовой базой знаний. Блок-схема алгоритма извлечения знаний из Neo4j представлена на рисунке 2.3.

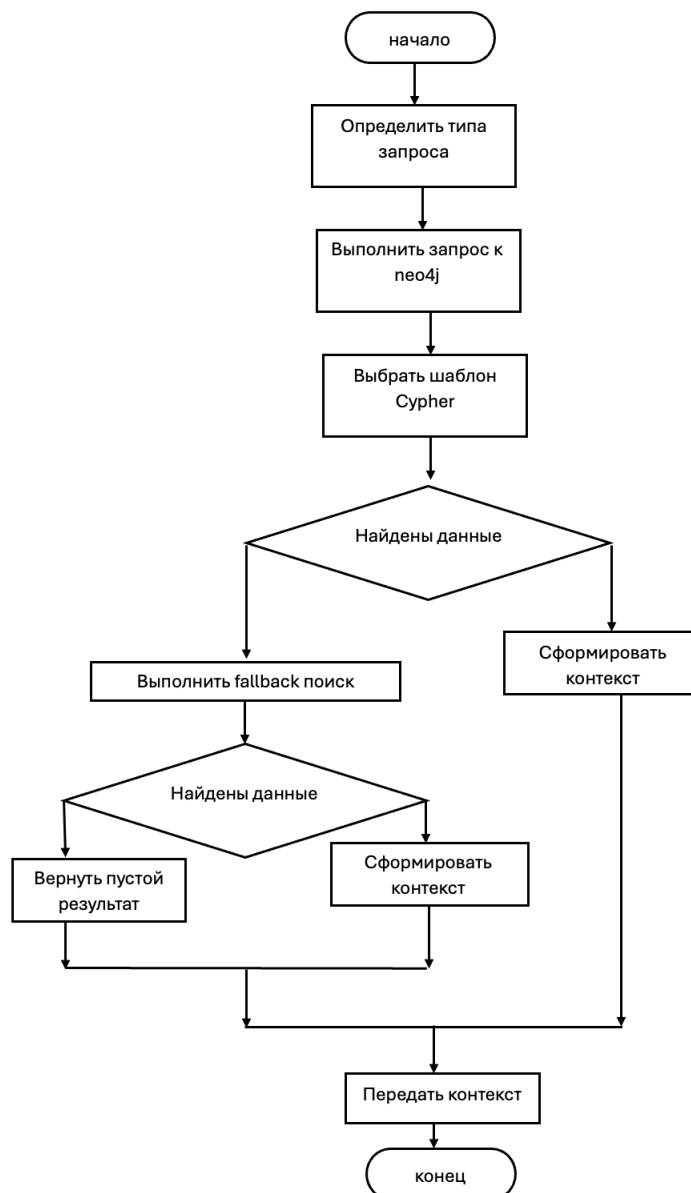


Рисунок 2.3 – Блок-схема алгоритма извлечения контекста из графовой базы знаний

Представленная схема показывает, что после определения типа запроса подсистема выбирает соответствующий шаблон обращения к базе знаний. Если релевантные данные найдены сразу, на их основе формируется контекст. Если же результат отсутствует, выполняется fallback-поиск, после которого либо формируется контекст, либо возвращается пустой результат. Затем полученный контекст передаётся во внешний модуль, использующий его для генерации ответа.

2.6 Обеспечение согласованности, расширяемости и сопровождаемости

Согласованность базы знаний обеспечивается как на уровне структуры данных, так и на уровне правил её наполнения. На структурном уровне ключевым механизмом выступают ограничения целостности, задаваемые в Neo4j. В текущем прототипе для каждого типа узлов используется ограничение уникальности атрибута `name`. Это позволяет предотвратить создание дублирующихся сущностей и обеспечивает стабильность ссылок на объекты при выполнении запросов.

Дополнительно согласованность поддерживается за счёт типизации отношений. Каждый тип связи применяется только в пределах тех сочетаний сущностей, для которых он имеет предметный смысл. Такой подход упрощает сопровождение графа и уменьшает вероятность семантически неверного наполнения.

Расширяемость базы знаний обеспечивается модульным характером её структуры. Добавление новых моделей, методов, задач, концептов и фреймворков не требует переработки уже существующего ядра. При расширении достаточно:

- создать новые узлы требуемых типов;
- установить между ними семантически корректные связи;
- при необходимости дополнить набор шаблонов Cypher-запросов.

Сопровождаемость достигается за счёт разделения физической структуры на отдельные файлы и ясного распределения ответственности между ними. Данные, ограничения и запросы не смешиваются в одном месте, а описываются отдельно. Это упрощает контроль изменений, делает структуру проекта более прозрачной и облегчает доработку базы знаний в дальнейшем.

Таким образом, принятые проектные решения позволяют развивать базу знаний без радикального изменения её основного каркаса, что особенно важно для предметной области искусственного интеллекта, которая быстро расширяется и регулярно пополняется новыми моделями, методами и инструментами.

2.7 Вывод по разделу проектирования

В результате проектирования была определена архитектурная роль базы знаний в составе интеллектуальной системы, выбран графовый формализм представления знаний и построена концептуальная модель предмет-

ной области, включающая основные сущности и связи между ними. В качестве технологической основы хранения принята графовая СУБД Neo4j, а в качестве языка обращения к данным — Cypher.

Разработанная структура базы знаний ориентирована на поддержку задач извлечения контекста в RAG-системе и включает как декларативный уровень представления фактов, так и процедурный уровень обработки запросов. Принятые решения по ограничениям целостности, типизации отношений и организации физической структуры хранения обеспечивают согласованность, расширяемость и сопровождаемость разрабатываемой базы знаний.

Следовательно, предложенная модель может служить устойчивой основой для дальнейшей реализации подсистемы извлечения знаний и последующего развития интеллектуальной справочной системы.